RIT جامعة روتشستر الأمريكية للتكنولوجيا في دبي
A Global American University in Dubai

| Department of Electrical Engineering and Computing Sciences | |
|---|---|
| **Academic Year** | 2025-2026 |
| **Semester** | ○ Fall ◉ Spring ○ Summer |
| **Course Code - Name** | **GCIS 124 - Software Development and Problem Solving II** |
| **Instructor** | Abinta Mir |
| **Exam** | ○ Quiz/Test ◉ Assignment ○ Mid ○ Fin ○ Other<br>Assignment 3 |
| **Duration** | 1 week |
| **Date** | Wednday, April , 2026 |

**CLOs:**

1. Utilize the basic constructs of a selected contemporary high-level programming language
2. Apply problem-solving skills to develop solutions for computing problems of varying complexity
3. Implement algorithms and data structures to solve computing problems of different difficulty levels.
4. Demonstrate fundamental software engineering concepts, including understanding needs, software design, solution testing, and incremental development.

**If this assessment maps to multiple CLOs, please complete the table below (comprehensive assessments only):**

| CLO | Question/ Component | Max Score | Students Score |
|---|---|---|---|
| 2,3 | Task 1 | 45 | |
| 4 | Task 2 | 10 | |
| 4 | Task 3 | 20 | |
| 4 | Task 4 | 25 | |
| | **Total Marks** | 100 | |

# Assignment 3: GreenPrint Networking

Spring 2026 - Software Development & Problem Solving II - GCIS-124

***GreenPrint — Part III***

*Client-Server Communication, Testing & Branding*

## Overview

In this assignment, you extend the GreenPrint ecosystem by wiring your Assignment 2 JavaFX GUI into a live TCP client-server architecture. Instead of a local countdown timer, the GUI (client) communicates with a dedicated GreenPrint Server that responds with a randomized discount between 1 % and 30 % applied to the session's total carbon emission score. This simulates a real-world incentive API where a backend rewards users for eco-friendly behaviour with dynamic offsets.

You will also write focused JUnit 5 tests for three specific classes introduced in this assignment, design your product's brand identity, and close with a short peer-feedback exercise conducted through a self-created Google Form shared during your final presentation.

## Objectives

Upon successful completion, students will be able to:

- Build a TCP client-server system in Java using Socket and ServerSocket
- Integrate network communication into an existing JavaFX GUI without blocking the UI thread
- Generate and transmit a randomized discount from server to client and apply it to live data
- Write targeted JUnit 5 unit tests for three newly introduced classes
- Design a professional brand identity for a software product
- Collect structured peer feedback using a self-created Google Form

# Task 1: Client-Server Discount System

This task replaces the eco-challenge timer from the original concept. The GUI you built in Assignment 2 becomes the client. A new standalone Java application acts as the server.

## A. Server Application — DiscountServer.java

Develop a standalone Java TCP server with the following behaviour:

- Listens on a port of your choice (e.g. 6000). Document your chosen port.
- Accepts one client connection at a time in a loop (a new connection per discount request is acceptable; concurrent multi-client support is optional).
- On each connection the server:
  - Reads the client's total emission value (a double sent as a String, e.g. "12.75") from the input stream.
  - Generates a random integer discount percentage between 1 and 30 inclusive using java.util.Random.
  - Calculates the discounted emission: discountedEmission = totalEmission * (1.0 - discount / 100.0).
  - Sends back a single response line in the format: DISCOUNT:15:10.84 — meaning discount in percentage (%), value after discount.
  - Closes the connection after the response.
- Prints a log line to the console for each transaction, e.g.: [SERVER] Client connected | Sent: 12.75 | Discount: 15% | Result: 10.84
- Wrap all socket operations in try-with-resources or try-catch-finally. The server must not crash on a broken connection.

> **Protocol summary:** Client sends: total emission as plain text (e.g. "12.75\n"). Server replies: "DISCOUNT:<pct>:<value>\n" (e.g. "DISCOUNT:15:10.84"). Connection closes after one exchange.

## B. Client Integration — GUI Update

Modify your Assignment 2 JavaFX application to act as the client:

- Add a new button labelled "Request Discount" on the Carbon Offset Transaction tab (or a new dedicated tab — your choice). The button is only enabled when at least one emission entry exists.
- When clicked, the GUI:
  - Reads the current total emission from FootprintTracker.getTotalEmissions().
  - Opens a TCP socket connection to the server (use configurable host/port — do not hard-code values in the UI logic; read them from a constant or make an additional properties file).
  - Sends the total emission value as a String followed by a newline.

  - Reads the server response and parses the discount percentage and discounted value.
  - Displays the result clearly in the GUI, e.g.: "Server discount applied: 15% — Your adjusted footprint is 10.84 kg $CO_2$ e"
  - Logs the transaction to greenprint_log.txt with operation type DISCOUNT_APPLIED.
- All socket I/O must run on a background thread (use Task<String> or Platform.runLater()). Never call blocking socket methods on the JavaFX Application Thread.
- If the server is unreachable, display a friendly inline error message. Do not crash or throw an unhandled exception.

### C. Error Handling Requirements

- Connection refused (server not running): show a Label with red text, e.g. "Could not reach server. Please try again later."
- Server sends malformed response: catch the parse error, show a warning, do not update the emission display.
- Network timeout: set a socket timeout (setSoTimeout) and handle SocketTimeoutException with a user-friendly message.

## Task 2: JUnit Testing

Write a JUnit test suite targeting one of the classes introduced in Assignment 3. These are the classes you created for the networking layer. You do NOT need to re-test model classes from Assignments 1 or 2 — the tests here must be specific to your new code.

### Choose from the Three Classes to Test

Your server and client introduce at least three new classes. Choose one and write a dedicated test class for each method in that class. Some of the propositions are:

| Class | Responsibility | Suggested Test Focus |
|---|---|---|
| DiscountCalculator | Computes discounted emission given a total and a percentage | Edge cases: 0%, 100%, 1%, 30%, negative input rejection |
| ResponseParser | Parses the server response string (e.g. "DISCOUNT:15:10.84") into structured data | Valid formats, malformed strings, missing fields, non-numeric values |
| ConnectionConfig | Holds and validates host/port configuration | Valid host, invalid port (0, 65536, negative), null host rejection |

*If your implementation uses different class names or splits responsibilities differently, you may test your own classe — but they must be Assignment 3 classes, not model classes from prior assignments.*

### Test Report

Include a brief test report to your presentation (0.5 page or a screenshot of the IDE test runner) showing all tests passing. If a test fails, explain why in a comment inside the test method.

## Task 3: Branding & Identity Design

Develop a professional brand identity for the GreenPrint product.

### A. Brand Package

- Product name: you may keep "GreenPrint" or invent your own. Justify the name briefly.
- Logo: design a simple, professional logo that visually represents carbon tracking or sustainability. Submit as PNG (minimum 400×400 px) and add to presentation.
- Motto / slogan: one short, memorable line (e.g. "Track Less. Live Greener.")
- Sales pitch: one paragraph explaining what makes the product effective and desirable for a university campus audience.

### B. Budget Planning

Estimate the cost breakdown for a real-world campus deployment. Present as a simple table with at least these four line items:

- Software development & testing (estimate hours × hourly rate)
- Server hosting for the discount/incentive service
- Marketing and campus outreach
- Annual maintenance and updates
- Explain impact of branding and marketing

# Task 4: Final Presentation & Peer Feedback

## A. Presentation

Prepare a slide deck (PowerPoint or Google Slides, maximum 15 minutes) covering:

- Project overview and evolution from Assignment 1 through 3
- Architecture diagram showing client, server, and GUI layers
- Live demo: launch the server, open the GUI, add entries, click Request Discount, show the result
- JUnit test results — show the test output
- Brand identity: logo, motto, sales pitch
- Lessons learned and what you would improve
- Answer questions about code (additional 2 minutes for Q&A)

## B. Peer Feedback via Google Form

At the end of your presentation, invite classmates to submit structured feedback through a Google Form you create yourself. This replaces a developer-run survey, it is an audience feedback tool for your final demo.

Instructions:

1. Create a Google Form (or an equivalent free tool such as Microsoft Forms or Typeform) before your presentation date.
2. The form must include at least 5 questions targeting audience impressions of your final product. Suggested questions:
   - How clear was the discount mechanism explained? (1–5 scale)
   - Did the live demo work smoothly? (Yes / Mostly / No)
   - How professional does the brand identity look? (1–5 scale)
   - Was the GUI intuitive to understand from the demo? (1–5 scale)
   - What one feature would you add to GreenPrint? (open text)
3. Display the form link and/or QR code on your final slide during the Q&A window.
4. After all presentations are complete, export the form responses and include a 1-page summary (charts or a table) in your submission. You need at least 5 responses.

> **Tip:** Google Forms auto-generates response charts. Screenshot the summary view and paste it into your submission document — no manual charting needed.

## Grading Rubric (100%)

| Category | Weight |
|---|---|
| DiscountServer: correct random discount, protocol, console logging | **25%** |
| GUI Client: Request Discount button, background thread, result display, error handling | **20%** |
| JUnit Tests: one class fully tested, parameterized test, exception test, test report, etc. | **10%** |
| Brand Identity: logo, motto, pitch, budget table, brand impact paragraph | **20%** |
| Final Presentation: 10+ slides, live demo, architecture diagram, lessons learned (15-17 minutes) | **20%** |
| Peer Feedback Form: 5+ questions, QR code/link shown, response summary submitted | **5%** |

## Deliverables

Submit a single ZIP file named GCIS124_LastName_Assignment3.zip containing:

- Java source files — server (DiscountServer.java and supporting classes) and updated client/GUI, organized in packages
- JUnit test classes (includes screenshot added to presentation)
- Branding package: logo PNG, motto/pitch document, and budget table
- Presentation slide deck (PPTX or Google Slides link)
- Google Form link and response summary (PDF or screenshot, minimum 5 responses)
- Updated UML diagram reflecting the networking classes added in Assignment 3

## Academic Integrity

All submissions must comply with RIT's Academic Integrity Policy. All code must represent original work with unique logic. Network code, test classes, and branding materials must be developed by the submitting student. All students should know answers to all parts of project unrelated of who developed which part and be ready to explain at the time of presentation. For wrong answers and not providing explanation, points will be deducted. Use of Generative AI tools is permitted only for branding and logo creation, however creativity will be graded, and similar plain and non-creative logos will receive low marks. If using AI, list the tool, and include the prompts used to the presentation.